

PYTHON DETAILED NOTES

1. History and purpose of Python

Python was created by Guido van Rossum, with its first release in 1991. Python's development has been guided by the principle of code readability and a focus on minimizing syntax complexity, making it a popular choice for both beginners and experienced programmers. The language was finally released in 1991. When it was released, it used a lot fewer codes to express the concepts, when we compare it with Java, C++ & C. Its design philosophy was quite good too. Its main objective is to provide code readability and advanced developer productivity. When it was released, it had more than enough capability to provide classes with inheritance, several core data types of exception handling and functions. The two of the most used versions has to Python 2.x & 3.x. For various purposes such as developing, scripting, generation, and software testing, this language is utilized. It is used for:

Web development (server-side),

Software development,

Mathematics,

System scripting.

- What can Python do?

Python can be used on a server to create web applications.

Python can be used alongside software to create workflows.

Python can connect to database systems. It can also read and modify files.

Python can be used to handle big data and perform complex mathematics.

Python can be used for rapid prototyping, or for production-ready software development

2. Python Input and Output

`print()` is used to display the output.

Examples :

```
name = "xyz"

print( "hello" ,name)


name = "xyz"

print(f" hello {name}" )
```

Usage of sep and end

Sep is used for assign a common separator between the words.

End is used to assign the value of the last value of the value as the one specified with end.

Example: print("One" , "Two" , "Three" , sep=" - ") # Output:

One-Two-Three

Print("Hello" , end=" ")

Print("World!")

input () is used to get the input from user.

Example :

input('Enter your name:')

Default type of the value received via input() is string. To parse the value as int/float/double, the respective methods can be used. **Built in functions like map(), filter can be used with input ().**

3.Understanding Hello World Program in Python

```
print( "Hello, World!" )
```

print() is a built-in Python function that tells the computer to show something on the screen.

Anything after `#` in a line is a comment. Python ignores comments when running the code, but they help people understand what the code is doing.

Any text included within `""" """` and `""" """` is also considered a comment

- **Indentation in Python**

In Python, Indentation is used to define blocks of code. It tells the Python interpreter that a group of statements belongs to a specific block. All statements with the same level of indentation are considered part of the same block. Indentation is achieved using whitespace (spaces or tabs) at the beginning of each line.

4. Pycharm installation and setup

Step 1: Install Python

- ◆ If not already installed:

Go to <https://www.python.org/downloads>

Download the latest version.

During installation, check the box: "Add Python to PATH" .

Verify installation:

`python --version`

Step 2: Install PyCharm

- ◆ **Option 1: For Windows/macOS**

Visit <https://www.jetbrains.com/pycharm/download>

Choose:

Community Edition (Free) – good for basic Python development

Download and run the installer.

Follow the installation wizard.

◆ **Option 2: For Linux (Ubuntu/Debian)**

Sudo snap install pycharm-community – classic

Step 3: Launch PyCharm

On first launch, you may be asked to:

Accept privacy terms.

Choose a UI theme (light/dark).

Install recommended plugins (you can skip).

Step 4: Create a New Python Project

Open PyCharm

Click New Project

Set a project location (e.g., ~/PyProjects/MyFirstApp)

Choose a Python interpreter:

Use existing one (from system or virtualenv)

Or click “Add Interpreter” → “New Virtual Environment”

Step 5: Create and Run a Python Script

Right-click your project folder → New → Python File

Name it (e.g., main.py)

Type your code:

```
print( "Hello, PyCharm!" )
```

Right-click the file → Run 'main'

5.PYTHON Variables

In Python, variables are used to store data that can be referenced and manipulated during program execution. Variables are the name assigned to a value. Data type is inferred by itself upon assigning the values.

- **Naming conventions** to name a variable in python is as below:

- (a) Variable names can only contain letters, digits and underscores (_).
- (b) A variable name cannot start with a digit.
- (c) Variable names are case-sensitive (myVar and myvar are different).
- (d) Avoid using Python keywords (e.g., if, else, for) as variable names.

- **Value Assignment**

Value can be assigned to a python variable using "=" operator. There are 3 types of variable assignment.

Basic Assignment

Example : x = 5

Dynamic Assignment

Example : x = 5

x = "hello"

Multiple Assignment

Example : x = y =5
x,y,z = 5, 2, "hello"

- **Typecasting a variable**

Typecasting refers to the conversion of a value in one data type to another. Built in functions are available in python for the conversion as below.

int()

float()

str ()

To get the type of variable as per the value assigned , type() method can be used.

- **Scope of variables**

A variable can be created in python with 2 scopes:

Local : Variables inside a function. No keyword to be used.

Global : Variables defined outside function and assigned inside function.

"global" keyword is used to define the global Variables.

- **del keyword**

"del" keyword is used to remove a variable from namespace. This, it helps with memory management.

Eg : del x

After deletion, trying to access the variable x results in a NameError, indicating that the variable no longer exists.

6. Operators in python

OPERATORS: These are the special symbols. Eg- + , * , /, etc.

OPERAND: It is the value on which the operator is applied.

Types of operators are :

- (a) **Arithmetic operators** (+, -, *, /, //, %)
- (b) **Relational operators** (<, >, <=, >=, ==, !=)
- (c) **Logical operators** (and, or, not)
- (d) **Bitwise operators**

&- bitwise and

| - bitwise OR

^ - bitwise XOR : returns 1 if corresponding bits are different.

~ - bitwise not : inverts the value

<< - bitwise left shift : It is used to shift the bits of a number left thereby multiplying or dividing the number by two respectively. It is

equivalent to $a \times (2 \text{ raised to } b)$, where a is the value, b is the number of values to be shifted.

$>>$ - right shift : it is used to shift the bits of a number right thereby multiplying or dividing the number by two respectively. It is equivalent to $a // (2 \text{ raised to } b)$, where a is the value, b is the number of values to be shifted.

(e) **Assignment operators** ($=, +=, -=, /=, //=, \%, **=, \&=, |=, ^=$)

(f) **Walrus operators** introduced from python 3.8($:=$ used to assign a value to a variable within an expression.

Eg : `a = [1, 2, 3, 4, 5]`

```
# walrus operator
while(x := len(a))>2:
    a.pop()
    print(x)
```

(g) **Identity operators** :

The Python Identity Operators are used to compare the objects if both the objects are actually of the same data type and share the same memory location. “is” , “is not” are the operators used.

Note : While comparing objects in Python, the users often gets confused between the Equality operator and Identity ‘is’ operator. The equality operator is used to compare the value of two variables, whereas the identities operator is used to compare the memory location of two variables.

(g) **Membership operators** : The Python membership operators test for the membership of an object in a sequence, such as strings, lists, or tuples. Python offers two membership operators to check or validate the membership of a value. “in, not in” is used for the membership operators.

(h) **Ternary operators** : The ternary operator in Python, also known as a conditional expression, provides a concise way to write a simple if-else statement on a single line. It’s syntax is as below:

Value_if_true if condition else value_if_false

Example:

Age = 20

Status = “Adult” if age >= 18 else “Minor”

Print(status)

- **Precedence and Associativity of Operators in Python**

In Python, operators have different levels of precedence, which determine the order in which they are evaluated. When multiple operators are present in an expression, the ones with higher precedence are evaluated first. In the case of operators with the same precedence, their associativity comes into play, determining the order of evaluation.

7. Python Keywords

Keywords in Python are reserved words that have special meanings and serve specific purposes in the language syntax.

- **Getting List of all Python keywords**

<pre>import keyword # printing all keywords at once using "kwlist()" Print("The list of keywords is: ") Print(keyword.kwlist)</pre>

- **Types of keywords**

- (a) **Value keywords:** True, False, None
- (b) **Operator keywords:** and, or, not, is, in
- (c) **Control flow keywords:** if, else, elif, for, while, break, continue, pass, try, except, finally, raise, assert
- (d) **Function and class :** def, return, lambda, yield, class
- (e) **Context management :** with, as
- (f) **Import and module:** import, from
- (g) **Scope and namespace :** global, non local
- (h) **Async programming:** async, await

- Difference Between Keywords and Identifiers**

Keywords	Identifiers
Reserved words in Python that have a specific meaning	Names given to variables, functions, classes, etc.
Examples: if, else, for, while	Examples: x, number, sum, result
Part of the Python syntax.	User defined.
They cannot be redefined or changed.	Can be defined and redefined by the programmer.
Cannot be used as variable names.	Can be used as variable names (if not a keyword)

- Difference Between Variables and Keywords**

Keywords	Identifiers
Reserved words with predefined meanings in Python.	Used to store data

Cannot modify	Can be created, modified, and deleted by the programmer
Used to define the structure of Python code.	Hold values that are manipulated in the program.
Fixed by Python language and cannot be altered.	Variable names must follow naming rules but are otherwise flexible

8.Datatypes

Python Data types are the classification or categorization of data items.

- (a) **Numeric Data Type** : It represents the data that has a numeric value.
- Int(positive/negative numbers without fraction or decimal) float(real number with a floating-point representation. It is specified by a decimal point. Optionally, the character e or E followed by a positive or negative integer may be appended to specify scientific notation.) complex (specified as (real part) + (imaginary part)j)

Eg:

a = 5

Print(type(a))

```
b = 5.0
```

```
Print(type(b))
```

```
c = 2 + 4j
```

```
Print(type( c))
```

- (b) **Sequence Data Types** : The sequence Data Type in Python is the ordered collection of similar or different Python data types. Sequences allow storing of multiple values in an organized and efficient fashion. String, List and Tuple are the sequence data types.
- (c) **Boolean Data Type** : Data type with 2 values - True and False . Returns bool for the type() method
- (d) **Set Data Type** : unordered collection of data types that is iterable, mutable, and has no duplicate elements. Undefined order of elements.

Example :

```
# initializing empty set
```

```
S1 = set()
```

```
S1 = set( "GreetingssForGreetingss" )
```

```
Print( "Set with the use of String: ", s1)
```

```
S2 = set([ "Greetingss" , "For" , "Greetingss" ])
```

```
Print( "Set with the use of List: ", s2)
```

```
For l in set1:

    Print(l, end="    ")

Print( "Greetingss" in set1)
```

- (e) **Dictionary Data Type** : A collection of data values stored as key value pair like a map is called dictionary. Each key-value pair in a Dictionary is separated by a colon : , whereas each key is separated by a 'comma' . Keys are supposed to be unique. dict () can be used for dictionary creation.

Example :

```
# initialize empty dictionary

d = {}

d = {1: 'Greetingss', 2: 'For', 3: 'Greetingss'}

print(d)

# creating dictionary using dict() constructor

d1 = dict({1: 'Greetingss', 2: 'For', 3: 'Greetingss'})

print(d1)

print(d[ 'name' ])


# Accessing a element using get

Print(d.get(3))
```

9. List, Tuple, Set, Dictionary

List

List is a built-in dynamic sized array (automatically grows and shrinks). We can store all types of items (including another list) in a list. Accessing items in List can be done directly using their position (index), starting from 0.

Example:

```
# List of integers
```

```
A = [1, 2, 3, 4, 5]
```

```
# List of strings
```

```
B = [ 'apple' , 'banana' , 'cherry' ]
```

```
# Mixed data types
```

```
C = [1, 'hello' , 3.14, True]
```

```
Print(a)
```

```
Print(b)
```

```
Print( c)
```

```
# create list from a tuple
```

```
A = list((1, 2, 3, 'apple' , 4.5))
```

```
Print(a)
```

```
# Create a list [2, 2, 2, 2, 2]
```

```
A = [2] * 5
```

```
A = [10, 20, 30, 40, 50]
```

```
# Access first element
```

```
Print(a[0])
```

```
# Access last element
```

```
Print(a[-1])
```

We can add elements to a list using the following methods:

Append(): Adds an element at the end of the list.

Extend(): Adds multiple elements to the end of the list.

Insert(): Adds an element at a specific position.

Removing Elements from List

We can remove elements from a list using:

- **remove():** Removes the first occurrence of an element.
- **pop():** Removes the element at a specific index or the last element if no index is specified.
- **del statement:** Deletes an element at a specified index.

Example:

```
A = [10, 20, 30, 40, 50]
```

```
# Removes the first occurrence of 30
```

```
a.remove(30)
print( "After remove(30):" , a)
```

```
# Removes the element at index 1 (20)

Popped_val = a.pop(1)

Print( "Popped element:" , popped_val)

Print( "After pop(1):" , a)


# Deletes the first element (10)

Del a[0]

Print( "After del a[0]:" , a)


# Iterating over the list

For item in a:

    Print(item)
```

Below are the methods used with list:

Append(): Adds an element to the end of the list.

Copy(): Returns a shallow copy of the list.

Clear(): Removes all elements from the list.

Count(): Returns the number of times a specified element appears in the list.

Extend(): Adds elements from another list to the end of the current list.

Index(): Returns the index of the first occurrence of a specified element.

Insert(): Inserts an element at a specified position.

Pop(): Removes and returns the element at the specified position (or the last element if no index is specified).

Remove(): Removes the first occurrence of a specified element.

Reverse(): Reverses the order of the elements in the list.

Sort(): Sorts the list in ascending order (by default).

Sample program to illustrate all the list functions:

```
myList = ['x','b','c','x','d']

#Adds a value in specified index
myList.insert(2,'y')

#Appends the value 'e' to the list
myList.append('e')

#Finds the index of provided value. If no such value is there, value error is returned
print(myList.index('d'))

#Removes element in specified index from the list. If no value is specified, last element of
list is removed
myList.pop(2)

#Copies the whole list to a new one
myCopyList = myList.copy()
```

```
#Sorts the list in ascending order
myList.sort()

#reverse=True returns the sorted list in descending order
myCopyList.sort(reverse=True)

#Reverses the list
myList.reverse()

#Returns the number of occurrences of given value is returned. If no such value is found,
then zero is returned
print(myList.count('x'))

#Removes the first occurrence of value specified in list
myList.remove('x')

print(myList)
```

Tuple

A Python tuple is an ordered, immutable collection of elements. Tuples are similar to lists in that they can store multiple items in a single variable, and items are accessed by integer indices starting from 0. Once a tuple is created, its elements cannot be changed, added, or removed. Below are the methods used for tuple creation

```
myTup = ()
print("Empty tuple is ",myTup)
```

```
# Using String
myTup1 = ('Greetingss', 'For')
print("Tuple using String is ", myTup1)

# Using List
myList = [1, 2, 4, 5, 6]
print("Conversion of list to tuple is ",tuple(myList))

# Using Built-in Function
myTup3 = tuple('Greetingss')
print("Tuple using built in function is ",myTup3)

#Create tuple with repetition
myTup4 = ('Example,')*5
print("Tuple using * is ",myTup4)
```

Basic operations that can be done with tuple are :

- **Accessing of Python Tuples**

We can access the elements of a tuple by using indexing and slicing, similar to how we access elements in a list. Indexing starts at 0 for the first element and goes up to n-1, where n is the number of elements in the tuple. Negative indexing starts from -1 for the last element and goes backward. A sample program that access the values of a tuple with index value, slicing, and unpacking the tuple are as below:

```
# Using String
myTup1 = ('Greetingss', 'For')
print("Tuple using String is ", myTup1)
#Access tuple using index value
print("First value of tuple is ",myTup1[0])
```

```
# Using List
myList = [1, 2, 4, 5, 6]
myTup2 = tuple(myList)

#Tuple can be unpacked to variables, provided, number of variables to unpack is
same as the size of tuple
a,b,c,d,e = myTup2
print(f"Unpacked tuple is {a},{b},{c},{d},{e}")
print("Conversion of list to tuple is ",myTup2)

#Access a range of elements using slicing
print("Accessed using slicing with index 1-3 and till 1 are ",myTup2[1:4],"and
",myTup2[:2])
```

- **Concatenation of Tuples**

Tuples can be concatenated using the + operator. This operation combines two or more tuples to create a new tuple. Only the same datatypes can be combined with concatenation, an error arises if a list and a tuple are combined. Below is the sample code of concatenation

```
myTup1 = ('Greetingss', 'For')

myList = [1, 2, 4, 5, 6]
myTup2 = tuple(myList)

#Concatenate the tuples using + sign
myConcatTup = myTup1 + myTup2
print(f"Concatenated tuple of {myTup1} and {myTup2} are {myConcatTup}")
```

- **Slicing of Tuple**

Creating another tuple from the original tuple is called Slicing. “print(tup[::-1])” is used to get the reverse of current tuple.

```
# Define a tuple
numbers = (10, 20, 30, 40, 50, 60, 70)

print("Original tuple:", numbers)

# Slice from index 1 to 4 (20, 30, 40)
print("numbers[1:4]:", numbers[1:4])

# Slice from beginning to index 3 (10, 20, 30)
print("numbers[:3]:", numbers[:3])

# Slice from index 3 to end (40, 50, 60, 70)
print("numbers[3:]:", numbers[3:])

# Slice with step of 2 (every second element)
print("numbers[::2]:", numbers[::2])

# Reverse the tuple
print("Reversed tuple:", numbers[::-1])
```

- **Deleting a Tuple**

Del keyword is used to delete a tuple.

A sample program illustrating the methods used in tuple are

```
mySampleTuple = (0,1,(3,6), [3,4],'Hello',[3,4])

print("Number of occurrence of Hello is ",mySampleTuple.count( 'Hello' ))

print("Occurrence of [3,4] in ",mySampleTuple," is ",mySampleTuple.count([3,4]))

print("Index of [3,4] in ",mySampleTuple.index([3,4]))

#Returns the index value of given input after provided index
print("Index value of [3,4] after 1 is ",mySampleTuple.index([3,4],1))
```

```
mySampleTuple = ('Greetingss', 'For', 'Greetingss', 'website')

"""Returns highest & lowest value in the tuple if it is a numeric value. If the contents
are strings, string with
maximum and minimum length
is taken. Type error is returned if both numeric and string values are present in
tuple"""

print("Longest word of the tuple is ",max(mySampleTuple))
print("Shortest word of the tuple is ",min(mySampleTuple))
```

Slicing :

```
myTup1 = ('Greetingss', 'For',"Greetingss","website")
```

```
#Method 1
```

```
print(myTup1[::-1])
```

```
#Method 2
```

```
print(tuple(reversed(myTup1)))
```

```
#Clearing a tuple
```

```
# initializing tuple
```

```
test_tup = (1, 5, 3, 6, 8)
```

```
# printing original tuple
```

```
print("The original tuple : " + str(test_tup))
```

```
# Clearing a tuple
```

```
# using Reinitialization + tuple()
```

```
test_tup = tuple()
```

```
# print result. Type cast to str so that type error doesn't throws
```

```
print("The tuple after clearing values : " + str(test_tup))
```

List Comprehension

Method to remove empty tuples from a list.

```
a = [(1,2), (4,5),(),(2,4)]
```

```
print([x for x in a if x])
```

or

```
print(list(filter(None,a)))
```

Set

A Set in Python is used to store a collection of items that is unordered, mutable, and that do not allow duplicate elements. Heterogeneous unique elements are permitted. Frozen sets in Python are immutable objects that only support methods and operators

```
#Basic syntax
```

```
s = {10,40,20}
```

```
print(s,type(s))
```

```
#Type casting from list or tuple
```

```
s1=set([1,2,4])
```

```
print(s1,type(s1))
```

```
tup = (1,2,3,4)
```

```
list1 =[3,4,5]
```

```
s.add(tup)
```

```
print("Adding tuple", tup,"to the set ", s ," using add method is ",s)
```

```
s.update(list1)
print("Adding List ",list1 , "to set ",s ," using update method is ",s)
```

```
s.clear()
print("Clearing set using clear method is ",s)
```

```
=====
=====
```

```
#difference doesn't overwrite the lefthand side while difference_update overwrites
left hand side
```

```
a = {1,2,3,4,5}
```

```
b= {2,5,4,3,1,8,9,10}
```

```
print(a,type(a))
```

```
#print("Difference between ",a," and ",b," is ",a.difference(b))
```

```
print("post", a)
```

```
a.difference_update(b)
```

```
print("a is ",a)
```

```
print("b is ", b)
```

```
#Checks for subset and returns True or False
```

```
print("checking if ",a," is subset of ",b,"is ",a.issubset(b))
```

```
#Checks for superset and returns True or False
```

```
print("checking if ",b," is superset of ",a," is ",b.issuperset(a))
```

```
#isDisJoint() returns true if no common elements are there
```

```
print("Checking if ",a," is disjoint of ",b," is ",a.isdisjoint(b))
```

```
#Removes 5 from set a if there. else, value error is thrown
```

```
a.remove(5)
```

```
#Sorts in ascending order. Reverse order is not possible in sets
print("Sorted set of ",b," is ",sorted(b))
```

```
=====
=====
```

```
#A frozen set
x = frozenset([1,2,4])
print(x,type(x))
```

Dictionary

Python dictionary is a data structure that stores the value in key: value pairs. Values in a dictionary can be of any data type and can be duplicated, whereas keys can't be repeated and must be immutable.

```
#Basic Syntax
d1 = {1: 'lion', 2: 'tiger', 3: 'cat'}
print(d1)

#Access using key
print("The value of dictionary with key 1 using get() is ",d1.get(1))
print("The value of dictionary with key 2 using [] is ",d1[1])

# create dictionary using dict() constructor
d2 = dict(a = "parrot", b = "peacock", c = "hen")
print("Dictionary before updating value using [] is ",d2)
```

```
d2['a'] = "Great"
print("Dictionary after updating value is ",d2)
d2['d']="cock"
print("Dictionary after updating new value is ",d2)
del d2['d']
print("Dictionary after deleting value using del is ",d2)
d2.pop('a')
print("Dictionary after deleting value using pop is ",d2)
key,value = d2.popitem()
print(f"Dictionary after deleting value using pop item is {key} ,{value}",d2)
d2.clear()
print("Dictionary after clear value is ",d2)
```

10. Conditional Statements

Conditional statements are used to execute something based on conditions. Below are the types of conditional statements available in python.

(a) **If statement**

If statement is used to perform an action only if a condition returns true. Otherwise, no action is performed.

Eg :

```
x = input("Enter a text\t")
if 'a' in x:
    print(x," is valid")
```

Short hand if : if 'a' in x: print(x," is valid")

(b) **If .. else statement**

“Else” allows us to specify a block of code that will execute if the condition(s) associated with an if or elif statement evaluates to False. Else block provides a way to handle all other cases that don't meet the specified conditions.

Eg:

<pre>x = input("Enter a text\t") if 'a' in x: print(x," is valid") else: print(x," is not valid")</pre>
Short hand if else: <pre>x = input("Enter a text\t") res = "valid" if 'a' in x else "invalid" print(res)</pre>

(c)**Elif statement**

It allows us to check multiple conditions , providing a way to execute different blocks of code based on which condition is true. Using elif statements makes our code more readable and efficient by eliminating the need for multiple nested if statements.

Eg:

```
x = input("Enter a text\t")
if 'a' in x:
    print(x," is first")
elif 'b' in x:
    print(x," is second")
else:
    print(x," is third")
```

(d) **Match Case**

Similar to Switch -case. _ is used instead of default keyword in other languages

```
x = input("Enter a bird name\t")
match x:
    case "parrot":
        print("I am a parrot . I speak")
    case "peacock":
        print("I am a peacock . I dance")
    case "cock":
        print("I am a hen . I wake you up")
```

```
case _:
    print("I am another bird")
```

11. Loops

Loops are used to perform some actions repeatedly. Python has below loop methods.

(a) **while loop**

While loop is used to execute the block of statements when a specific condition is satisfied.

Eg:

```
i=0
c=0
a = 'obsqurazone'
print(len(a))
while i < len(a):
    print(a[i])
    if a[i]=='a' or a[i]=='e' or a[i]=='i' or a[i]=='o' or a[i]=='u':
        c=c+1
    i=i+1
```

```
print("Number of vowels are ",c)
```

If an action need to be done if the condition is not satisfied, while..else can be used. Here, only “hi” will be printed as c is set as 10

```
c = 10
while(c<5):
    print("hello")
    c+=1
    break
else:
    print("hi")
```

Infinite while loops can occur if the condition always returns True. Hence, the statement in such loops would execute till memory is full.

(b) **for loop**

Python For Loops are used for iterating over a sequence like lists, tuples, strings, and ranges.

Eg:

```
a = 'obsqurazone'
```

```
#Iterate over each index and get it printed in seperate
for i in a:
    print(i)
#for value in range(start,stop,step)
for i in range(0,7,2):
    print(i)
#for value in range(start,stop)
for i in range(7,2):
    print(i)
#for value in range(stop)
for i in range(5):
    print(i)
```

(c) **Nested loops**

We can use while and for loop inside each other. This is called nested looping. Here is an example of printing multiplication table upto 10 for 0 till 3.

```
# Running outer loop from 2 to 3

for i in range(4):

    # Printing inside the outer loop
    # Running inner loop from 1 to 10
    for j in range(1, 11):

        # Printing inside the inner loop
        print(i, "*", j, "=", i*j)
    # Printing inside the outer loop
    print()
```

(d) **Loop Control statements**

Loop control statements are used to control the flow of code.

The continue statement skips the current iteration and proceeds to the next iteration of the loop.

The pass statement is a null operation; it does nothing when executed. It's useful as a placeholder for code that you plan to write in the future.

The break statement is used to exit the loop prematurely when a certain condition is met.

```
print("Example of break:")
for i in range(1, 6):
    if i == 3:
        print("Break at i =", i)
```

```
        break # Exit loop when i == 3
    print(i)

print("\nExample of continue:")
for i in range(1, 6):
    if i == 3:
        print("Continue at i =", i)
        continue # Skip the rest of the loop when i == 3
    print(i)

print("\nExample of pass:")
for i in range(1, 4):
    if i == 2:
        pass # Does nothing, placeholder
        print("Pass at i =", i)
    else:
        print(i)
```

12. Python Functions

A block of statements used to perform a specific action is called a function. The action in the method can be re-used so that duplication of logic would be removed.

- **Types of functions**

There are 2 types of functions in python.

- (a) **Built-in library functions**

Few built in library functions are :

Function	Description
map()	Returns an object of results after applying to a function.
max()	Finds largest of the collection data
Min()	Finds smallest of the collection data
Next()	To move to the next value of iterator
Pow()	Finds the power of a number
Print()	Prints the output
Input()	Takes input from user
Filter()	Filters the given sequence with the help of a function that tests each element in the sequence to be true or not

(b) **User defined functions**

Functions declared, defined and invoked by user is called user define functions. Function declaration has below format

```
def function_name(parameter):
    #Function Body
    Return statement.
```

Types of user defined functions are as below :

i. **Parameterized functions**

Functions that accept arguments/functions to process and return result is called parameterized functions.

```
#Parameterised functions without return value
def printName(name):
    print(name)
#Function call
printName("Obsqura")
```

ii. **Functions with default arguments**

A default value can be assigned to the parameter of function as below.

```
#Default parametered functions
def findSum(x,y=50):
    print(x+y)
findSum(10)
```

iii. **Keyword argument functions**

Value of arguments can be passed using the keywords. Here , keyword does not mean the reserved words. Eg:

```
#Keyword argument functions
def findDiff(a,b):
    print(a-b)

findDiff(a=20,b=10)
```

iv. Variable length argument functions.

For the instances where number of argument is unknown, *args
/**kwargs can be passed as parameter as below:

```
#Variable length argument functions
def findMax(*args):
    print(max(args))

findMax(10,20,60,32)
findMax(20,90,0)
findMax(80)
```

v. Functions with return value

Functions that use return keyword to return the result is called functions with return values.

```
#Functions with return value
def findMin(*args):
    return min(args)
print(findMin(10,20,30,40))
```

vi. **Anonymous functions**

Functions without name is called anonymous function. Lambda keyword is used to define such methods. It is useful for the operations that can be done in single line .

```
#Takes only 1 input. Hence, one variable on left end
res = lambda x : x**2
print(res(4))
#Takes 2 inputs . Hence, x,y is passed
result = lambda x,y : x*y
print(result(2,5))
```

- **Python Decorators**

Decorators are a powerful and flexible way to modify or extend the behaviour of functions or methods, without changing their actual code. It is commonly used in logging, authentication etc. Syntax of python decorators are as below

```
def decorator_name(func):
    def wrapper(*args, **kwargs):
        # Add functionality before the original function call
        result = func(*args, **kwargs)
        # Add functionality after the original function call
        return result
    return wrapper

@decorator_name
def function_to_decorate():
    # Original function code
    pass
```

Types of decorators based on the functionality are :

a. Function Decorators

Commonly used decorators. Takes function as input and returns another new function

```
#Name of outside function is used as the tag of function to be
invoked with inner function.
def simple_decorator(func):
    def wrapper():
        print("Before the function runs")
        func()
        print("After the function runs")
    return wrapper
```

```
# Using the decorator
@simple_decorator
def say_hello():
    print("Hello, World!")

say_hello()
```

b. Method Decorators

A method decorator is similar to a function decorator but specifically applies to methods within a class. The main difference is that the method has an implicit first argument (self for instance methods or cls for class methods), which needs to be handled in the decorator. A sample code is given below :

```
def method_decorator(func):
    def wrapper(self, *args, **kwargs):
        print(f"Before calling {func.__name__} method")
        result = func(self, *args, **kwargs)
        print(f"After calling {func.__name__} method")
        return result
    return wrapper

class MyClass:
    @method_decorator
    def greet(self, name):
```

```
print(f"Hello, {name}!")

obj = MyClass()
obj.greet("Alice")
```

c. Class Decorators

A class decorator is used to modify or enhance the behavior of a class. Class decorators take a class as an argument and can modify its attributes, methods, or behavior. A sample code is given below :

```
def class_decorator(cls):

    cls.extra_attribute = "This is added by the decorator"

    return cls

@class_decorator

class MyClass:

    def greet(self):

        print("Hello!")
```

```
obj = MyClass()

print(obj.extra_attribute) # Output: This is added by the decorator
```

d. Property Decorators

In Python, the `@property` decorator is used to define a getter method as a property. It allows you to define methods that can be accessed as attributes. You can also define a setter or deleter method with `@<property_name>.setter` and `@<property_name>.deleter`.

```
class MyClass:

    def __init__(self, value):

        self._value = value


    @property
    def value(self):

        return self._value


    @value.setter
    def value(self, new_value):

        if new_value < 0:

            raise ValueError("Value must be non-negative")

        self._value = new_value
```

```
obj = MyClass(10)

print(obj.value) # Calls the getter method


obj.value = 20 # Calls the setter method

print(obj.value)


try:

    obj.value = -5 # This will raise a ValueError

except ValueError as e:

    print(e)
```

e. Parameterized Decorators

A parameterized decorator is a decorator that accepts its own arguments. This allows you to customize its behavior. Essentially, it's a decorator that returns another decorator.

```
def repeat_decorator(times):

    def decorator(func):

        def wrapper(*args, **kwargs):

            for _ in range(times):

                func(*args, **kwargs)

            return wrapper

        return decorator

    return decorator
```

```
@repeat_decorator(times=3)

def say_hello():

    print("Hello!")

say_hello()
```

f. Decorator with arguments

A decorator with arguments allows you to pass additional parameters into the decorator function, which can then be used to modify the behavior of the wrapped function. This is different from parameterized decorators because here the arguments are passed directly when calling the decorator.

```
def greet_decorator(message):

    def decorator(func):

        def wrapper(*args, **kwargs):

            print(message)

            return func(*args, **kwargs)

        return wrapper

    return decorator

@greet_decorator("Hello, how are you?")

def greet():
```

```
print("I am fine, thank you!")

greet()
```

g. Built-in Decorators

Python provides several built-in decorators, such as:

- **@staticmethod:** Used to define a static method that doesn't require an instance.
- **@classmethod:** Used to define a class method that receives the class itself as the first argument (cls).
- **@property:** Used to define getter methods that can be accessed like attributes.
- **@functools.lru_cache:** Used to cache results of function calls, often used for optimization.
- **@functools.wraps:** A helper function to ensure the decorated function retains the original function's metadata.

13. Strings

In Python, a string is a sequence of Unicode characters. It is used to store textual data, such as names, sentences, or any combination of letters, digits, and symbols.

- i. Create a String variable

```
s = "Hello, world!"
```

- ii. String Operations
- iii. String Indexing and Slicing
- iv. String Methods
- v. String Comparison

14. Python OOPs Concept

Object Oriented Programming is a fundamental concept in Python. Below are the OOP concepts

a) **Objects**

Objects can be considered to be actual entities that exist and interact within a program. It represents a specific implementation of the class and holds its own data.

An object consists of:

- State: It is represented by the attributes and reflects the properties of an object.
- Behavior: It is represented by the methods of an object and reflects the response of an object to other objects.
- Identity: It gives a unique name to an object and enables one object to interact with other objects.

An object can be created using format below. It is also called object instantiation

Objectname = Classname()

b) **Class**

Collection of objects are called class. A class can have a set of attributes, methods that performs some action. Keyword class is used to create a class. Attributes of class would be in public access mode and is accessible with a dot (.) operator. A class would have a constructor with it. Constructor is used to initialize the attributes of an object. `__init__()` acts as the constructor in python. It is mandatory to use parameter `self` as the first parameter of constructor / class methods so that the instance attributes access is made possible. Function in the class can be marked static and access it directly using class name with `@staticmethod`. 2 types of variables are used in a class

i. Class Variables

Variables shared across all the instance of class is called class variables. It is defined outside any methods . All objects of the class share the same value for a class variable unless explicitly overridden in an object.

ii. Instance Variables

These are defined within `__init__()` or other instance methods. Every object created carry its own copy of instance variables, independent of other objects.

A sample program using class and instance variables are below given

```
class Car:
    #wheel is the class variable
    wheel = "4"
    # Here, make and model are the instance variables.
    def __init__(self, make, model, year):
        self.make = make
        self.model = model
        self.year = year

    def __del__(self):
        print('Destructor called, Car data deleted.')
```

```
benz = Car("Benz", "BMW", 1999)
print(benz.wheel," ",benz.make," ",benz.model," ",benz.year)
bmw = Car("BMW", "BMW", 2000)
print(bmw.wheel," ",bmw.make," ",bmw.model," ",bmw.year)
benz.year = 2001
print("After update of instance variable year in benz is ",benz.year)
print("After update of instance variable year in benz, bmw has year
value of ",bmw.year)
#When the class variable is modified, its value is changed across the
objects created
Car.wheel = 5
print("After update of class variable wheel in benz is ",benz.wheel)
print("After update of class variable wheel in bmw is ",bmw.wheel)

del benz
```

c) **Inheritance**

Inheritance allows a class (called a child or derived class) to inherit attributes and methods from another class (called a parent or base class). This promotes code reuse, modularity, and a hierarchical class structure. If the child doesn't have a constructor declared, constructor of parent class is inherited by itself. Super() is used to call the methods of parent class. Below are the types of inheritance in python

i. **Single Inheritance**

Inheritance level where only one parent and child class are inherited. A sample program is given below.

```
class Animal:
    def __init__(self):
        print('Animal is created')
    #Here, makeSound() has no change in parameters. Each child class
    overrides its implementation.
    def makeSound(self):
        print("Animal is making sound")

class Dog(Animal):
    def __init__(self):
        super().__init__()
        print('Dog is created')
    def makeSound(self):
        print("Dog is barking")

animals = [Dog(),Animal()]
for animal in animals:
    animal.makeSound()
```

ii. Multiple Inheritance

Inheritance type where child class inherits from more than one parent class. Here, to achieve the values of parent class via constructors we need to follow format:

ParentClass.__init__(self,attributes)

A sample program is given below

```
class Employee:
```

```

def __init__(self,name,age):
    self.name=name
    self.age=age
class Job:
    def __init__(self,salary):
        self.salary=salary

class Department(Employee,Job):
    def __init__(self,name,age,salary,department):
        Employee.__init__(self,name,age)
        Job.__init__(self,salary)
        self.department=department

dep = Department("Sam",23,20000,"accounts")
print(dep.name," ",dep.age," ",dep.salary," ",dep.department)

```

iii. Multi-level Inheritance

Child class is derived from a class which is also derived from another class. A sample program is given below

```

class Employee:
    def __init__(self,name,age):
        self.name=name
        self.age=age
class Job(Employee):
    def __init__(self,name,age,salary):
        super().__init__(name,age)
        self.salary=salary
class Department(Job):
    def __init__(self,name,age,salary,department):
        super().__init__(name,age,salary)

```

```
self.department=department

dep = Department("Sam",23,20000,"accounts")
print(dep.name," ",dep.age," ",dep.salary," ",dep.department)
```

iv. Hierarchical Inheritance

Multiple classes inherit from a single parent class. A sample class is given below

```
class Employee:
    def __init__(self,name,age):
        self.name=name
        self.age=age
class Salary(Employee):
    def __init__(self,name,age,salary):
        super().__init__(name,age)
        self.salary=salary
class Department(Employee):
    def __init__(self,name,age,salary,department):
        super().__init__(name,age)
        self.department=department

'''Here, Employee acts as the parent. Salary, Department are different
child classes.

Hence, salary cannot be retrieved by Department and viceversa'''
dep = Department("Sam",23,20000,"accounts")
sal = Salary("Sam",23,20000)
print(dep.name," ",dep.age," ",dep.department)
print(sal.name," ",sal.age," ",sal.salary)
```

--

v. Hybrid Inheritance

Combination of one or more inheritance results in hybrid mode. A sample code is given below

```
class Salary:
    def __init__(self,salary):
        self.salary = salary

class Manager(Salary):
    def __init__(self,salary, name,department):
        self.name = name
        self.department = department

class SeniorManager(Manager,Salary):
    def __init__(self,salary,name,department):
        Manager.__init__(self,salary,name,department)
        Salary.__init__(self,salary)

'''Here, Manager is inherited from Salary. Hence, single level inheritance is
acheived.
Senior Manager inherits from Salary and Manager and achieved multiple
inheritance'''
senior = SeniorManager(20000,"lia","accounts")
print(senior.name," ",senior.salary," ",senior.department)
```

d) Polymorphism

Polymorphism refers to the ability to take more than one form. A python function can have same name, but different behaviours-based context. It is made possible via method overloading and overriding. There are 2 types of polymorphism.

i. **Compile Time Polymorphism**

It determines polymorphism during the compilation of program. Here, methods/ operators can have same name but behave different based on the input parameters. This is called function overloading. It can be done inside one class only. Python does not support method overloading by default. If you define multiple methods with the same name, only the latest definition will be used. A sample program is given below:

```
import math

class CalculateArea:

    def findArea(self,*args):
        if len(args) == 1:
            return math.pi*args[0]*args[0]
        elif len(args) == 2:
            return args[0] * args[1]
        else:
            print("Invalid number of arguments")

calc = CalculateArea()
```

```
print("Circle area is ",calc.findArea(4))  
print("Rectangle area is ",calc.findArea(4,7))
```

ii. Run-Time Polymorphism

When the behaviour of a method need to be determined at run time based on the object passed, run time polymorphism is adapted. This technique is called method overriding. We can override a method in parent class to child class.

```
class Animal:  
    #Here, makeSound() has no change in parameters. Each child class  
    overrides its implementation.  
    def makeSound(self):  
        print("Animal is making sound")  
class Dog(Animal):  
    def makeSound(self):  
        print("Dog is barking")  
  
class Cat(Animal):  
    def makeSound(self):  
        print("Cat is meowing")  
  
animals = [Dog(),Cat(),Animal()]  
for animal in animals:  
    animal.makeSound()
```

e) Encapsulation

Encapsulation refers to the bundling of data (attributes) and methods (functions) that operate on the data into a single unit. It also involves restricting direct access to the internal state of an object from outside the class, controlling how the data can be accessed and modified. Data members of a class is distinguished as public, private and protected. In python, public members are identified by seeing that no `'_'` is present for the members. Hence, default access modifier in python is public.

Protected members are identified by single `'_'` ahead. Private members are enforced to have double `'__'` in the front. Below given is the example of encapsulating data member to the getter (get_attribute) and setter(set_attribute) methods. Here, constructor is private, other variables are public.

```
class Sum:
    sum = 0
    num1 = 0
    num2 = 0
    def __init__(self):
        print("Object Created")
    def get_num1(self):
        return self.num1
    def get_num2(self):
        return self.num2

    def set_num1(self,num1):
        self.num1 = num1
    def set_num2(self,num2):
        self.num2 = num2
sum = Sum()
sum.set_num1(5)
```

```
sum.set_num2(6)
print(sum.get_num1())
print(sum.get_num2())
```

A sample code of accessing the public, private and protected variables are given below

```
# program to illustrate access modifiers of a class

# super class
class Super:

    # public data member
    var1 = None

    # protected data member
    _var2 = None

    # private data member
    __var3 = None

    # constructor
    def __init__(self, var1, var2, var3):
        self.var1 = var1
        self._var2 = var2
        self.__var3 = var3

    # public member function
    def displayPublicMembers(self):
```

```
# accessing public data members
print("Public Data Member:", self.var1)

# protected member function
def _displayProtectedMembers(self):

    # accessing protected data members
    print("Protected Data Member:", self._var2)

# private member function
def __displayPrivateMembers(self):

    # accessing private data members
    print("Private Data Member:", self.__var3)

# public member function
def accessPrivateMembers(self):

    # accessing private member function
    self.__displayPrivateMembers()

# derived class
class Sub(Super):

    # constructor
    def __init__(self, var1, var2, var3):
        Super.__init__(self, var1, var2, var3)

    # public member function
    def accessProtectedMembers(self):

        # accessing protected member functions of super class
        self._displayProtectedMembers()
```

```
# creating objects of the derived class
obj = Sub("Greetingss", 4, "Greetingss!")

# calling public member functions of the class
obj.displayPublicMembers()
obj.accessProtectedMembers()
obj.accessPrivateMembers()
print()

# Can also be accessed using
obj._displayProtectedMembers()
obj._Super__displayPrivateMembers()
print()

# Object can access protected member
print("Object is accessing protected member:", obj._var2)
print("Object is accessing private member:", obj._Super__var3)

# object can not access private member, so it will generate Attribute error
# print(obj.__var3)
```

f) **Data Abstraction**

Data abstraction refers to the process of hiding unnecessary details and displaying the needed data. In python, it is achieved using abstract classes and methods. Abstract method is not a default feature in python. “ABC and “abstractmethod” need to be imported from ABC. @abstractmethod decorator is used for creating

abstract method. Concrete methods are the methods defined in an abstract base class with their complete implementation. Concrete methods are required to avoid replication of code in subclasses. An **abstract class** is simply a class that **cannot be instantiated directly**, but it can still include **regular methods** (with full implementations).

```
from abc import ABC, abstractmethod
class Car(ABC):

    def __init__(self, brand,model,year):
        self._brand = brand
        self._model = model
        self._year = year
        self.engine_Started = True

    def startEngine(self):
        if self.engine_Started:
            print("Engine Started")
        else:
            print("Engine not Started")
    #Annotation defines that printDetails() is the abstract method
    @abstractmethod
    def printDetails(self):
        pass

    #Concrete methods are set
    def accelerate(self):
        print("Increasing speed")

    def brake_applied(self):
        print("Car Stopped")
```

```
#Create a child class
class HatchBack(Car):
    #Abstract method is added with the logic in extended class
    def printDetails(self):
        print("Brand: ",self._brand)
        print("Model: ",self._model)
        print("Year: ",self._year)

car1 = HatchBack("Hyundai","Eon","2020")
car1.accelerate()
car1.brake_applied()
car1.printDetails()
```

Iterator

An iterator is an object which contains a countable number of values and it is used to iterate over iterable objects like list, tuples, sets, etc . Using an iterator-

- iter() function is used to create an iterator containing an iterable object.
- next() function is used to call the next element in the iterable object.
- After the iterable object is completed, to use them again reassign them to the same object.

15. Exception Handling

Python Exception Handling handles errors that occur during the execution of a program. Exception handling allows to respond to the error, instead of crashing the running program.

- Differences between error and exception**

Error	Exception
A fundamental problem, often a syntax or logic mistake, that prevents the program from running or behaving as intended.	An event that occurs during the execution of a program that disrupts the normal flow of instructions. It can be handled to prevent program termination.
Typically detected during the parsing/compilation phase (e.g., <code>SyntaxError</code>) or due to severe system issues (e.g., <code>OutOfMemoryError</code>).	Occurs during runtime when unexpected conditions arise (e.g., <code>ZeroDivisionError</code> , <code>FileNotFoundError</code> , <code>ValueError</code>).
Generally not recoverable within the program's normal flow; requires code correction or system resource management.	Can be caught and handled using try-except blocks, allowing the program to continue execution or gracefully exit.
Often due to programmer mistakes in code structure, syntax, or fundamental logic.	Can be caused by external factors (e.g., missing files, network issues) or invalid data input, in addition to programming errors.
<code>SyntaxError</code> , <code>IndentationError</code> , <code>SystemError</code> (e.g., <code>MemoryError</code>).	<code>TypeError</code> , <code>NameError</code> , <code>ZeroDivisionError</code> , <code>FileNotFoundError</code> , <code>ValueError</code> , <code>IndexError</code> .
Requires fixing the underlying code issue or system resources.	Handled using try, except, else, and finally blocks to manage the program's response to the event.

addressing system-level problems.

- **Try,except,finally,else, raise an exception**

In Python, try, except, else, and finally are keywords used for exception handling, allowing for robust program execution by managing potential errors.

The try block contains the code that might raise an exception. Python attempts to execute the code within this block.

The except block is executed if an exception occurs within the try block. It specifies how to handle a particular type of exception or any exception if no specific type is mentioned. Multiple except blocks can be used to handle different types of exceptions.

The else block is optional and is executed only if no exception occurs within the try block. It allows for code to be run specifically when the try block executes successfully.

The finally block is also optional and is always executed, regardless of whether an exception occurred in the try block or was handled by an except block. It is typically used for cleanup operations, such as closing files or releasing resources, ensuring these actions happen even if an error disrupts normal execution.

Raise Keyword is used to raise exceptions or errors. The raise keyword raises an error and stops the control flow of the program. It is used to bring up the current exception in an exception handler so that it can be handled further up the call stack.

An example of using raise is given below:

```
a = 5

if a % 2 != 0:

raise Exception("The number shouldn't be an odd integer")
```

Below is the code sample using try,except,finally and else.

```
try:
    # Code that might raise an exception
    result = 10 / 0 # This will raise a ZeroDivisionError
    print(result)
except ZeroDivisionError:
    # Code to handle the specific exception
    print("Exception: Cannot divide by zero!")
else:
    print("No exception found")
finally:
    # Code that always executes
    print("This code in the finally block will always run.")

print("Program continues after the try-except-finally block.")
```

- **Types of exceptions and common exceptions in python**

a) Built-in Exceptions

Most common exceptions provided by python.

Exception Name	Description
BaseException	The base class for all built-in exceptions.
Exception	The base class for all non-exit exceptions.
ArithmeticError	Base class for all errors related to arithmetic operations.
ZeroDivisionError	Raised when a division or modulo operation is performed with zero as the divisor.
OverflowError	Raised when a numerical operation exceeds the maximum limit.

	of a data type.
FloatingPointError	Raised when a floating-point operation fails.
AssertionError	Raised when an assert statement fails.
AttributeError	Raised when an attribute reference or assignment fails.
IndexError	Raised when a sequence subscript is out of range.
KeyError	Raised when a dictionary key is not found.
MemoryError	Raised when an operation runs out of memory.

NameError	Raised when a local or global name is not found.
OSError	Raised when a system-related operation (like file I/O) fails.
TypeError	Raised when an operation or function is applied to an object of inappropriate type.
ValueError	Raised when a function receives an argument of the right type but inappropriate value.
ImportError	Raised when an import statement has issues.

ModuleNotFoundError	Raised when a module cannot be found.
---------------------	---------------------------------------

b) Exception for specific conditions

It is used to handle specific conditions in the code.

Examples of Specific Condition Exceptions:

NameError: Raised when a variable or function is used before being defined.

- o **Specific Condition:** This happens only when Python can't find a variable in the current scope, i.e., the name of the variable has not been declared yet.

NotImplementedError: Raised when an abstract method or function has been defined, but the code does not implement it.

- o **Specific Condition:** You use this error when creating an abstract base class or interface, signaling that a certain method must be implemented by subclasses, but hasn't been yet.

RecursionError: Raised when the maximum recursion depth is exceeded.

- o **Specific Condition:** This occurs when your recursive function keeps calling itself infinitely without hitting a base case, causing a stack overflow.

TimeoutError: Raised when an operation times out.

- o **Specific Condition:** This happens when an I/O operation or network request takes too long and exceeds the allowed time limit.

c) Custom Exceptions

Custom exceptions in Python are user-defined exception classes that allow for more specific and tailored error handling within an application. They enhance code clarity and maintainability by providing meaningful error messages and enabling granular control over different error scenarios.

Defining a Custom Exception:

To define a custom exception, create a new class that inherits from Python's built-in Exception class or one of its subclasses (e.g., ValueError, TypeError).

```
#Defining a custom exception
class MyCustomException(Exception):

    def __init__(self, message):
        self.message = message
        super().__init__(self.message)

def process_exception(a,b):
    if b <0:
        raise MyCustomException("Division by negative is not
allowed")
    elif b == 0:
        raise MyCustomException("Division by zero is not
allowed")
    else:
        print(a/b)
#Adding try-except is helps with handling the exception and
```

```
avoid tracebacks printed in output
try:
    process_exception(10, 6)
except MyCustomException as e:
    print(e.message)
#Here, else block executes when no exceptions are caught
else:
    print("No exception found")
#Finally block prints anyway
finally:
    print("Closing program")
```

d) Context manager related exceptions

Context Manager-related exceptions refer to exceptions that are raised in the context of using a context manager—typically when managing resources like files, network connections, or database transactions. In Python, context managers are commonly used with the with statement, which ensures that resources are properly cleaned up after they are no longer needed, even if an error occurs during execution.

The two main components that help manage these resources are:

`__enter__()` method: This sets up the resource or context.

`__exit__()` method: This is called to clean up the resource, even if an exception was raised

Here is an example of context related exceptions

```
import time
class TimeoutManager:
```

```

def __init__(self, timeout):
    self.timeout = timeout

def __enter__(self):
    print("Entering timeout operation")
    return self

#exc_type return the exception type (class) that was raised inside the
with block, or None if no exception occurred.
#exc_val returns the exception instance (object) raised. None if no
exception occurred
#traceback helps with retrieving detailed information on where
exception is raised. None if no exception
def __exit__(self, exc_type, exc_val, traceback):
    if exc_type == TimeoutError:
        print("Timeout at",exc_val)
        return True
    return False
"""If an exception is raised within the with block, the context manager's
__exit__() method is responsible for handling that exception.
If the exception is not handled, it will propagate after the __exit__()
method is done."""
with TimeoutManager(5):
    time.sleep(5)
    raise TimeoutError("Operation timed out")

```

16. File Handling

File handling refers to the process of performing operations on a file such as creating, opening, reading, writing and closing it, through a programming interface. It involves managing the data flow between the program and the file system on the storage device, ensuring that data is handled safely and efficiently.

To open a file we can use `open()` function, which requires file path and mode as arguments. Below are the modes available

Mode	Description	Behavior
r	Read-only mode.	Opens the file for reading. File must exist; otherwise, it raises an error.
rb	Read-only in binary mode.	Opens the file for reading binary data. File must exist; otherwise, it raises an error.
r+	Read and write mode.	Opens the file for both reading and writing. File must exist; otherwise, it raises an error.
rb+	Read and write in binary mode.	Opens the file for both reading and writing binary data. File must exist; otherwise, it raises an error.
w	Write mode.	Opens the file for writing. Creates a new file or truncates the existing file.
wb	Write in binary mode.	Opens the file for writing binary data. Creates a new file or truncates the existing file.
w+	Write and read mode.	Opens the file for both writing and reading. Creates a new file or truncates the existing file.
wb+	Write and read in binary	Opens the file for both writing and reading

	mode.	binary data. Creates a new file or truncates the existing file.
a	Append mode.	Opens the file for appending data. Creates a new file if it doesn't exist.
ab	Append in binary mode.	Opens the file for appending binary data. Creates a new file if it doesn't exist.
a+	Append and read mode.	Opens the file for appending and reading. Creates a new file if it doesn't exist.
ab+	Append and read in binary mode.	Opens the file for appending and reading binary data. Creates a new file if it doesn't exist.
x	Exclusive creation mode.	Creates a new file. Raises an error if the file already exists.
xb	Exclusive creation in binary mode.	Creates a new binary file. Raises an error if the file already exists.
x+	Exclusive creation with read and write mode.	Creates a new file for reading and writing. Raises an error if the file exists.
xb+	Exclusive creation with read and write in binary mode.	Creates a new binary file for reading and writing. Raises an error if the file exists.

Below operations can be performed with files in python

a) Opening a file

Below format can be used to open the file.

```
file = open("Greetingss.txt", "r")
```

b) Reading a file

To read a file, `read()` can be used. To read a file successfully, the mode should have `r/w+/r+/rb` etc in it.

```
file = open("Greetingss.txt", "r")  
  
content = file.read()
```

c) Writing to a file

Writing to a file is done using `file.write()` which writes the specified string to the file. If the file exists, its content is erased. If it doesn't exist, a new file is created.

```
try:  
    #Write to the file  
    file = open("files.txt", "w")  
    content = file.write("Hello World")  
    print(content)  
    file.close()  
    #Appends Hello World Trial to the last content of file with 'a' mode  
    file = open("files.txt", "a")  
    content = file.write("\nHello World Trial")  
    file.close()
```

```
files = open("files.txt", "r")
content = files.read()
print(content)
except FileNotFoundError:
    print("File not found")
else:
    file.close()
```

d) Closing a file

Closing a file is essential to ensure that all resources used by the file are properly released. `file.close()` method closes the file and ensures that any changes made to the file are saved.

`with` statement is used for resource management. It ensures that file is properly closed after its suite finishes, even if an exception is raised. `with open()` as method automatically handles closing the file once the block of code is exited, even if an error occurs. This reduces the risk of file corruption and resource leakage.

```
with open("Greetingss.txt", "r") as file:

    content = file.read()

print(content)
```

e) Handling exception when closing a file

File handling is done as below :

```
try:

    file = open("Greetingss.txt", "r")

    content = file.read()

    print(content)

finally:

    file.close()
```

Github Integration and jenkins

Install git
git init
git clone<repository-url>
git status
git add .
git commit -m "Initial commit"
git remote add origin <a href="https://github.com/<username>/<repository-name>.git">https://github.com/<username>/<repository-name>.git
git remote -v
git branch -M main
git push -u origin main
git pull --rebase origin main
git push [If ! [rejected] main -> main (fetch first) upon]

If error is “{Rejected: non-fast-forward” , git fetch origin git rebase origin/main git push
If error is “Rejected after rebase / amended commit” , git push --force-with-lease
If error is “Rejected: remote contains work you don’ t have” , git pull --rebase git push

Integrating Jenkins with pytest allows you to automate the execution of your tests and generate reports as part of your CI/CD pipeline. This integration helps you run your tests whenever there are changes in your code repository, automatically triggering tests, collecting results, and notifying you about failures or successes.

Step 1 : Download Jenkins war .

Step 2 : Open cmd and change directory to the downloaded location.

Step 3 : Provide command **java -jar Jenkins.**

A password will be given upon running the above command. Copy it for the registration purpose.

Search for the port in which Jenkins is started. 8080 is the common port.

Open localhost:8080 and register using the password copied. Create a username and password.

New Item->Enter project name ->Free style project

Add parameter

Source Code Management -> Git -> Add github URL-> Select branch specifier

Triggers ->Build Periodically ->Add schedule as "0 17 * * 1-5"

Add build steps as below

```
python --version
```

```
cd BasicPython
```

```
python runner.py
```

Save and build